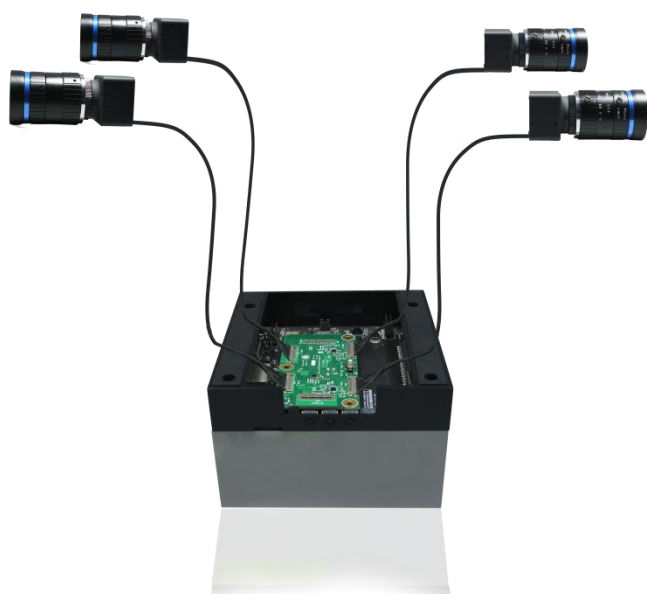


e-CAM82\_CUOAGX

# GStreamer Usage Guide



Version 1.3

e-con Systems

3/10/2023



**e-con Systems**

Think Camera. Think e-con.

### **Disclaimer**

e-con Systems reserves the right to edit/modify this document without any prior intimation of whatsoever.

# Contents

|                                  |           |
|----------------------------------|-----------|
| <b>INTRODUCTION TO GSTREAMER</b> | <b>3</b>  |
| <b>SETUP PROCEDURE</b>           | <b>4</b>  |
| <b>TESTED GSTREAMER EXAMPLES</b> | <b>6</b>  |
| <b>TROUBLESHOOTING</b>           | <b>10</b> |
| <b>WHAT'S NEXT?</b>              | <b>11</b> |
| <b>GLOSSARY</b>                  | <b>12</b> |
| <b>SUPPORT</b>                   | <b>13</b> |

# Introduction to GStreamer

---

GStreamer is a powerful and flexible multimedia framework with a lot of capabilities. It supports various features such as capturing, displaying, encoding, and decoding of image data. While using GStreamer, the possibilities of manipulating data are limitless.

The commands and output messages in this manual are represented by different colors as shown in below table.

**Table 1: Notation of Colors**

| Color  | Notation                              |
|--------|---------------------------------------|
| Blue   | Commands running in development board |
| Red    | Output message in development board   |
| Orange | Commands running in Client PC         |

This document explains how to install GStreamer-1.0 on the Jetson AGX Orin™/Xavier™ development kit and use it with the available plugins.

# Setup Procedure

---

This section describes how to verify the setup before testing the GStreamer pipelines.

To know the details of available plugins, please refer to the *Accelerated GStreamer* section in [NVIDIA Jetson Linux Developer Guide](#)

The steps to verify the setup before testing GStreamer pipelines are as follows:

1. Run the following commands to install GStreamer-1.0 binaries and libraries on the Jetson AGX Orin™/Xavier™ platform.

```
$ sudo apt-add-repository universe
$ sudo apt-get update
$ sudo apt-get install gstreamer1.0-tools
gstreamer1.0-alsa gstreamer1.0-plugins-base
gstreamer1.0-plugins-good gstreamer1.0-plugins-bad
gstreamer1.0-plugins-ugly gstreamer1.0-libav
```

**Note:** If the installation process stops with **could not get lock /var/lib/dpkg/lock** message, run the following command to remove the file and proceed with the installation.

```
$ sudo rm /var/lib/dpkg/lock
```

2. Run the following command to check the GStreamer-1.0 version.

```
$ gst-inspect-1.0 --version
```

The output message appears as shown below.

```
gst-inspect-1.0 version 1.16.3
GStreamer 1.16.3
https://launchpad.net/distros/ubuntu/+source/gstreamer
1.0
```

**Note:** Make sure that e-CAM82\_CUOAGX is connected, and the required drivers are loaded.

During booting, the module drivers for e-CAM82\_CUOAGX will be loaded automatically in the Jetson AGX Orin™/Xavier™ development kit.

3. Run the following command to confirm whether the camera is initialized.

```
$ sudo dmesg | grep -i "Detected eimx485 sensor"
```

The output message appears as shown below.

```
Detected eimx485 sensor
```

The output messages indicate that all the cameras are initialized properly.

4. Run the following command to check the presence of video nodes.

```
$ ls /dev/video*
```

The output message for four camera setup appears as shown below.

```
/dev/video0  
/dev/video1  
/dev/video2  
/dev/video3
```

**Note:** The number of video node entries and number of connected cameras must be same.

# Tested GStreamer Examples

---

This section describes some of the tested GStreamer commands which work on the Jetson AGX Orin™/Xavier™ development kit.

**Note:** Please run the following commands to change the power mode to maximum for better performance and to get maximum frame rate.

```
$ sudo nvpmodel -m 0
$ sudo jetson_clocks
$ sudo $HOME/max-isp-vi-clks.sh
```

## Example 1: Streaming 1080p from a single camera (HW accelerated).

Run the following command to stream 1080p video from a single camera.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=<n> !
"video/x-raw(memory:NVMM),
width=(int)1920,height=(int)1080, format=(string)NV12" !
nv3dsink -e
```

**Note:** Replace <n> with the number of video device node, from which you need to stream.

## Example 2: Streaming 4K resized to 1080p from a single camera (HW accelerated).

Run the following command to stream 4K video from a single camera.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=<n> !
"video/x-raw(memory:NVMM),
width=(int)3840,height=(int)2160, format=(string)NV12" !
nvvidconv left=8 right=1928 top=8 bottom=1088 ! "video/x-
raw(memory:NVMM), width=(int)1920,height=(int)1080,
format=(string)NV12" ! nv3dsink -e
```

**Note:** Replace <n> with the number of video device node, from which you need to stream.

## Example 3: Record 1080p in H.264 format to a video file (HW accelerated).

Run the following command to record 1080p video in H.264 format to a video file.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=<n> !
"video/x-raw(memory:NVMM),
width=(int)1920,height=(int)1080, format=(string)NV12" !
nvv4l2h264enc ! avimux ! queue ! filesink
location=<filename>.mkv
```

**Note:** Replace <n> with the number of video device node, from which you need to stream. Change <filename> with a name (For example, Video\_1), to which the video will get stored.

**Example 4: Record 4K in H.264 format to a video file (HW accelerated).**

Run the following command to record 4K video in H.264 format to a video file.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=<n> !  
"video/x-raw(memory:NVMM), width=(int)3840,  
height=(int)2160, format=(string)NV12" ! nvv4l2h264enc !  
avimux ! queue ! filesink location=<filename>.mkv
```

**Note:** Replace **<n>** with the number of video device node, from which you need to stream. Change **<filename>** with a name (For example, Video\_1), to which the video will get stored.

**Example 5: Playback of saved 4k and 1080p video file (HW accelerated).**

Run the following command to playback the saved video.

```
$ gst-launch-1.0 filesrc location=<filename>.mkv !  
avide mux ! h264parse ! nvv4l2decoder ! nv3dsink
```

**Note:** Change **<filename>** with the name under which the video is recorded

**Example 6: Playback of 4K video file (HW accelerated).**

Run the following command to playback the saved video.

```
$ gst-launch-1.0 filesrc location=<filename>.mkv !  
avide mux ! h264parse ! nvv4l2decoder ! nvvidconv !  
"video/x-raw(memory:NVMM), width=(int)1920,  
height=(int)1080, format=(string)NV12" ! nv3dsink
```

**Note:** Change **<filename>** with the name under which the video is recorded

**Example 7: Record 4K from two cameras and save as separate video files.**

Run the following command to record video in 4K resolution from two cameras and save as separate video files.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=0 !  
"video/x-raw(memory:NVMM), width=(int)3840,  
height=(int)2160, format=(string)NV12" ! nvv4l2h264enc !  
avimux ! queue ! filesink location=<file1>.mkv  
nvarguscamerasrc sensor-id=1 ! "video/x-  
raw(memory:NVMM), width=(int)3840, height=(int)2160,  
format=(string)NV12" ! nvv4l2h264enc ! avimux ! queue !  
filesink location=<file2>.mkv
```

**Note:** Change **<file1>** and **<file2>** with names (For example, Video\_1, Video\_2), to which the videos will get stored.

**Example 8: Record 1080p from two cameras and save as separate video files.**

Run the following command to record video in 1080p resolution from two cameras and save as separate video files.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=0 !
"video/x-raw(memory:NVMM), width=(int)1920,
height=(int)1080, format=(string)NV12" ! nvv4l2h264enc !
avimux ! queue ! filesink location=<filename>.mkv
nvarguscamerasrc sensor-id=1 ! "video/x-
raw(memory:NVMM), width=(int)1920, height=(int)1080,
format=(string)NV12" ! nvv4l2h264enc ! avimux ! queue !
filesink location=<filename>.mkv
```

**Note:** Change <file1> and <file2> with names (For example, Video\_1, Video\_2), to which the videos will get stored.

#### Example 9: Network streaming H.264 encoded FHD video using RTP over UDP (HW accelerated).

Run the following command to stream the video captured by camera connected with Jetson AGX Orin™/Xavier™ development board.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=<n> sensor-
mode=0 ! "video/x-raw(memory:NVMM),
width=(int)3840,height=(int)2160, format=(string)NV12" !
nvvidconv ! "video/x-raw(memory:NVMM),
format=(string)NV12, width=(int)1920, height=(int)1080" !
nvv4l2h264enc insert-vui=1 ! h264parse ! rtph264pay
config-interval=1 ! udpsink host=<ip_address>
port=<port_no>
```

**Note:**

- Replace <n> with the number of video device node, from which you need to stream, <ip\_address> with the IP address of the Client device (For example, 192.168.6.100) and <port\_no.> with the port number of the Client device (For example, 5000).
- Do not close the application in Jetson AGX Orin™/Xavier™ development board.

Run the following command in the Client device to view the video.

```
$ gst-launch-1.0 udpsrc port=<port_no>
caps="application/x-rtp, media=(string)video, encoding-
name=H264, sprop-parameter-
sets=(string)\ "Z0JAKJZUA8ARPyzUAAADAQAAAABAAADADyPCIRq\\
,aM48gA\\=\\" payload=(int)96" ! rtph264depay !
h264parse ! decodebin ! videoconvert ! autovideosink
sync=false
```

**Note:** Replace <port\_no> with the port number of the Client device (For example, 5000).

#### Example 10: Capturing 4K still image.

Run the following command to capture 4K still image.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=<n> num-
buffers=1 ! "video/x-raw(memory:NVMM),
```

```
format=(string)NV12, width=(int)3840, height=(int)2160" !  
nvjpegenc ! filesink location=<filename>.jpg
```

**Note:** Replace **<n>** with the number of video device node, from which you need to stream. Change **<filename>** with a name (For example, Capture\_1), to which the image will get stored.

Run the following command to capture 1080p still image.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=<n> num-  
buffers=1 ! "video/x-  
raw(memory:NVMM),format=(string)NV12, width=(int)1920,  
height=(int)1080" ! nvjpegenc ! filesink  
location=<filename>.jpg
```

**Note:** Replace **<n>** with the number of video device node, from which you need to stream. Change **<filename>** with a name (For example, Capture\_1), to which the image will get stored.

# Troubleshooting

---

In this section, you can view the commonly occurring issue and its troubleshooting step.

**During network streaming, face issues in video quality and frame rate.**

Network streaming quality and frame rate depends on the network bandwidth and receiver decode capability. In the receiver end, if you use software decoder, the frame rate and quality will get affected. Therefore, avoid using software decoder in the receiver end to resolve this issue.

# What's Next?

---

After understanding the usage of GStreamer application, you can refer to the following documents to understand more about e-CAM82\_CUOAGX.

- *e-CAM82\_CUOAGX Datasheet*
- *e-CAM82\_CUOAGX Release Package Manifest*

# Glossary

---

**4K:** 4K resolution (Industry name for 3840 x 2160 resolution).

**FHD:** Full HD (Industry name for 1920 x 1080 resolution).

**IP:** Internet Protocol.

**RTP:** Real-time Transport Protocol.

**UDP:** User Datagram Protocol.

# Support

---

## **Contact Us**

If you need any support on e-CAM82\_CUOAGX product, please contact us using the Live Chat option available on our website - <https://www.e-consystems.com/>

## **Creating a Ticket**

If you need to create a ticket for any type of issue, please visit the ticketing page on our website - <https://www.e-consystems.com/create-ticket.asp>

## **RMA**

To know about our Return Material Authorization (RMA) policy, please visit the RMA Policy page on our website - <https://www.e-consystems.com/RMA-Policy.asp>

## **General Product Warranty Terms**

To know about our General Product Warranty Terms, please visit the General Warranty Terms page on our website - <https://www.e-consystems.com/warranty.asp>

## Revision History

| Rev | Date        | Description                     | Author          |
|-----|-------------|---------------------------------|-----------------|
| 1.0 | 09-Nov-2022 | Initial Draft                   | Camera Dev Team |
| 1.1 | 16-Nov-2022 | Added support for Xavier        | Camera Dev Team |
| 1.2 | 22-Feb-2023 | 4-lane support                  | Camera Dev Team |
| 1.3 | 10-Mar-2023 | Updated Setup Procedure section | Camera Dev Team |